CSE422 : Artificial Intelligence
Project Report
Project Title : Stroke Prediction

Group No :4 , CSE422 Lab Section :06, Fall 2023	
ID	Name
20101009	TAMEEM ISLAM
21101032	DIPRO NISHANTO
21201340	NOSHIN ANJUM MALIK PIASHA
21201283	ABU SAYED

Table of Contents

Introduction

In the wake of a personal tragedy that struck my family two years ago, the term "stroke" has resonated profoundly in my mind. The passing of my uncle, attributed to a stroke, has been a haunting experience, prompting me to channel grief into a meaningful endeavor. In honor of his memory and with a deep-seated commitment to raising awareness about the debilitating impacts of strokes, I, Tameem, along with a dedicated group, embarked on a journey to develop a machine learning model for stroke prediction.

Our motivation stems from a genuine desire to prevent others from succumbing to the unforeseen consequences of this medical condition. Recognizing the significance of early intervention in mitigating the severity of strokes, we have harnessed the power of machine learning to create a predictive model. This model relies on robust algorithms meticulously crafted from extensive datasets encompassing the essential personal statistics of individuals.

Through our project, we aspire to provide a tool that empowers individuals and healthcare professionals with accurate predictions. By leveraging the insights gleaned from our machine learning model, we aim to contribute to a future where strokes can be anticipated and prevented, ushering in an era of proactive healthcare. The ultimate goal is to transform the tragedy that touched our lives into a catalyst for positive change, ensuring that others may lead healthier and safer lives.

DATASET DESCRIPTION

SOURCE:
https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset

DATASET DESCRIPTION:

***There are 12 features in this project including

1.id

2gender

3.age

4.hypertension

5.heart_disease

6.ever_married

7.work_type

8.Residence_type

9.avg_glucose_level

10.bmi

11.smoking_status

12.stroke

***Classification problem

This problem set is a classification problem. According to the definition of classification, unique values that are not continuous, like True or False, Male or Female, etc., are predicted or categorized using classification algorithms. In order to calculate continuous values like price, income, age, etc., regression techniques are used. Since we are only

dealing with discontinuous data in this problem—such as gender or female and yes/no about whether or not the stroke is occurring—we have chosen to utilize the classification problem. Therefore, as this is a classification problem, no continuous data of any type is required.

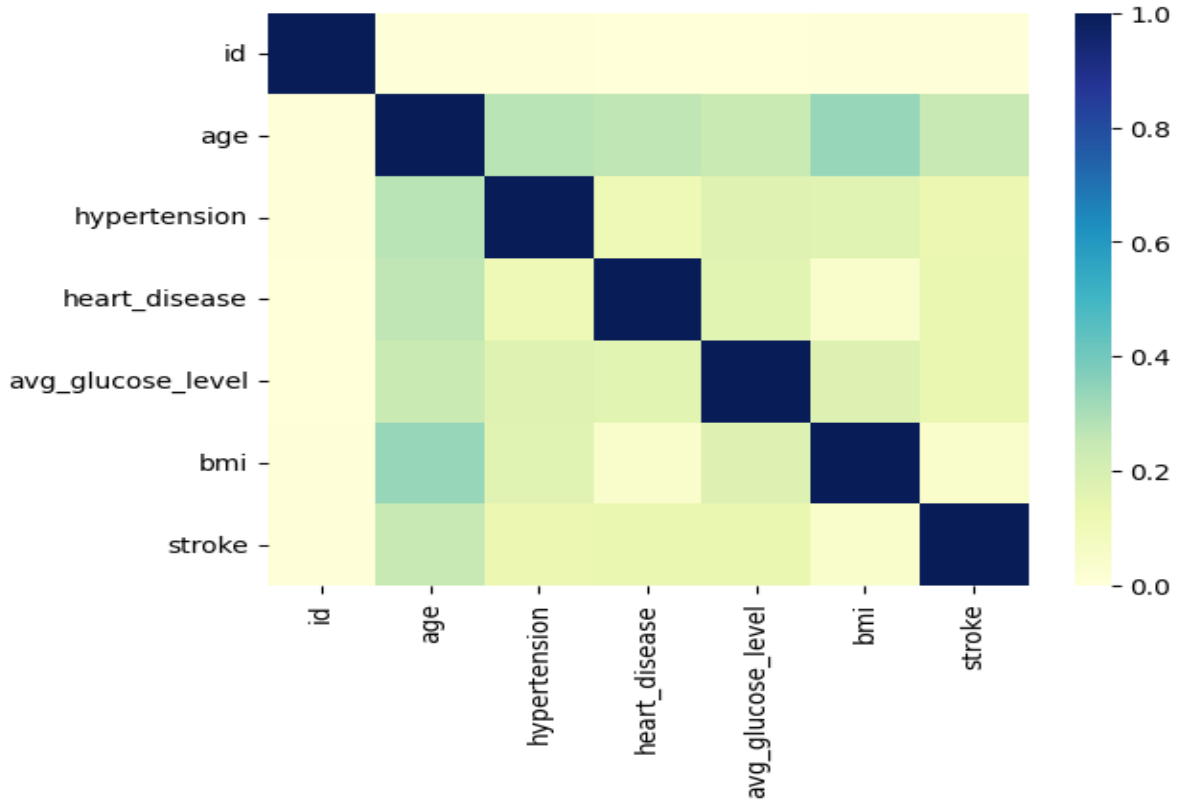
***Data points

The total data points are 5110.

***Categorical or Quantitive:

While quantitative data compares distinct groups or measures numerical values across time, categorical data describes characteristics of a population based on non-numeric variables. For example, gender, group is a categorical value which has no numeric number. However, age and weight are regarded as quantitative facts. This dataset's features include a combination of quantitative and categorical data. Since we obtain non-numerical values for the features gender, ever married, work type, and residence type, these are categorical datasets instead of to the quantitative datasets for id, age, bmi, hypertension, heart disease, and average glucose level. so it is a mixture of both categories.

***Correlation of features:

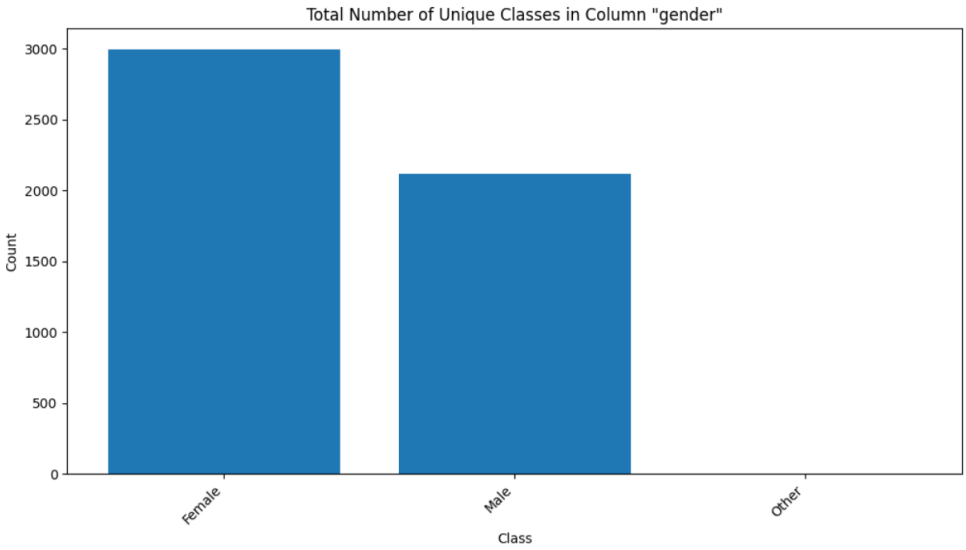


It's a statistical relationship between pairs of variables. In the input and output features, correlation analysis can help identify how changes in one variable are associated with changes in another. If we take a look at our heatmap, there's a value, and the color range ratio is given from 0.0 to 1.0 (light to darker color). We can see that the correlation between two of the same variables is the same and is pointed out in dark blue. We will not work with that value for our model generation purposes. but it is the highest value. There's no correlation between any variables and id because id cannot define anyone's age,gender, disease conditions, and all because it is a random value. We can see the strongest correlation between age and BMI, followed by age, hypertension, and age-related heart_disease. The correlation between avg glucose level and hypertension, heart disease, BMI, and stroke is also weak. We remove the value if the correlation between two variables is larger than

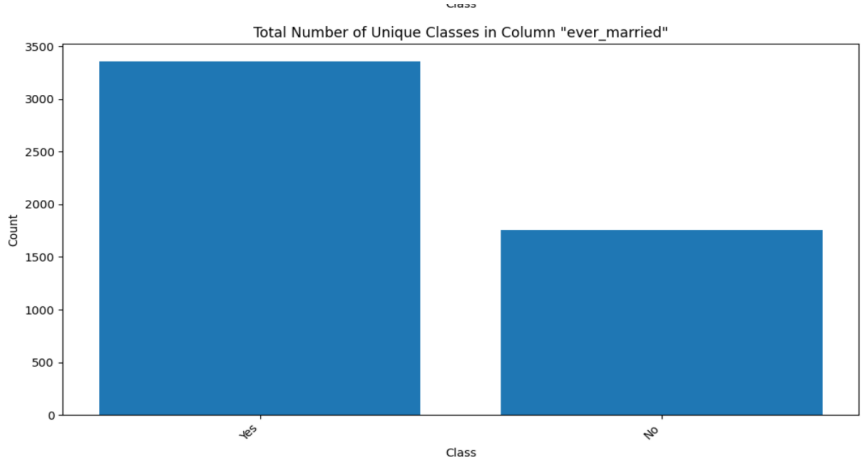
0.75 at least twice. But since the majority of the relationships here fall between 0.0 and 0.6, we don't need to do that.

***Imbalanced Dataset:

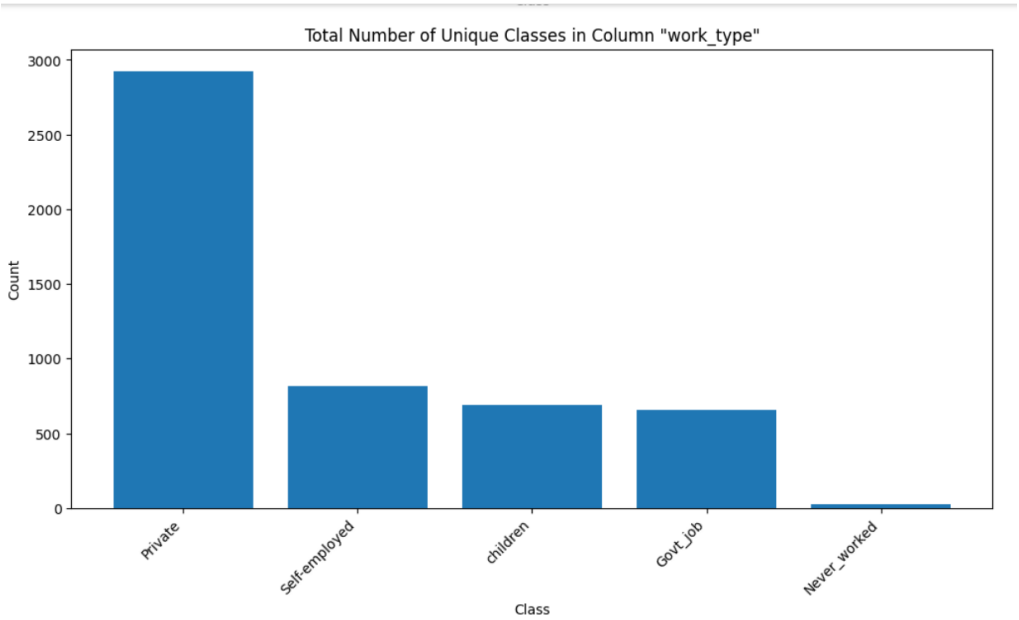
Our dataset is a mixture of both balanced and imbalanced data set. As we can see the following graphs below:



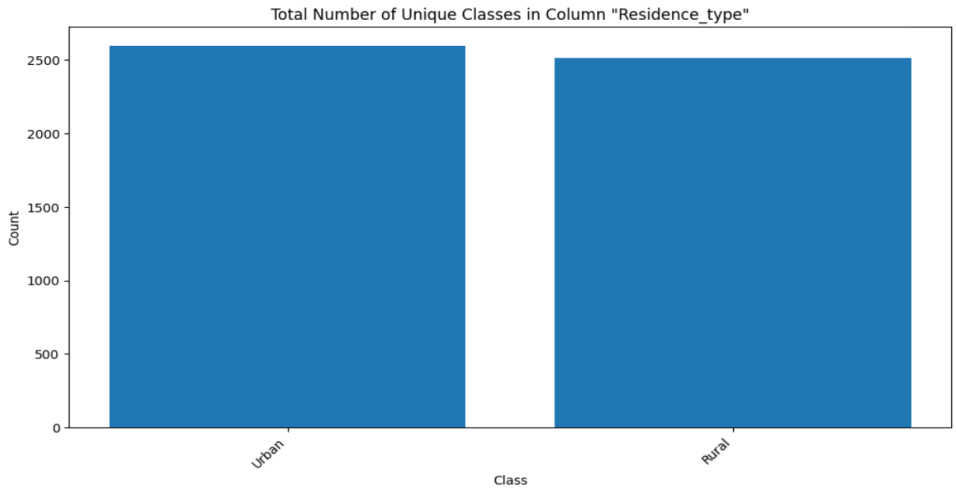
Given that the female to male ratios are not equal, this data set is imbalanced.



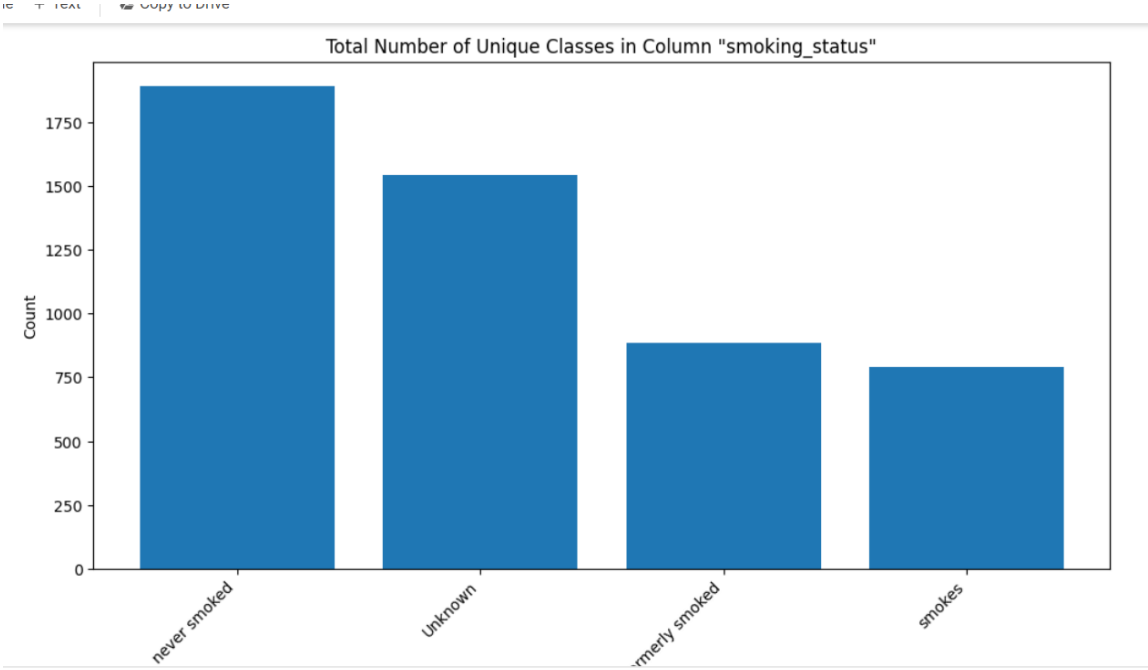
Given that the total number of ever_married ratios are not equal, this data set is imbalanced.



This is the mixture of both balanced and imbalanced dataset as the same ratios of self employed, children and govtjob persons bar chart is shown. So this part is Balanced or slightly imbalanced. But the comparison with private job holders and the others is huge. So it is imbalanced.



The graph showing the similarity between those who live in urban and rural areas indicates that this group is balanced.



This set also combines elements of balance and imbalance. Since it symbolizes the vast number of people who have never smoked. Others are not as large as this, which is why there is an imbalance. The number of smokers who have smoked in the past is equal, meaning that they are balanced with each other.

DATASET PREPROCESSING

Faults:

1.NULL value: We have only one null value in a column name ‘BMI’. By using isna.sum() we found null valued column.

```
1 stroke.isnull().sum()
```

```
id                   0
gender               0
age                  0
hypertension         0
heart_disease        0
ever_married         0
work_type            0
Residence_type       0
avg_glucose_level    0
bmi                201
smoking_status       0
stroke               0
dtype: int64
```

2. <u>Categorical values:</u> We found lots of categorical values that are not numerical numbers

```
[ ]  1 stroke.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5110 entries, 0 to 5109
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   id                 5110 non-null   int64  
 1   gender             5110 non-null   object 
 2   age                5110 non-null   float64
 3   hypertension       5110 non-null   int64  
 4   heart_disease      5110 non-null   int64  
 5   ever_married       5110 non-null   object 
 6   work_type          5110 non-null   object 
 7   Residence_type     5110 non-null   object 
 8   avg_glucose_level  5110 non-null   float64
 9   bmi                5110 non-null   float64
 10  smoking_status     5110 non-null   object 
 11  stroke             5110 non-null   int64  
dtypes: float64(3), int64(4), object(5)
memory usage: 479.2+ KB
```

<u>Solutions:</u>We don't have to delete any row or column values. But here we can use impute values to solve the issue.

<u>Impute values:</u>

Imputation is the process of replacing null values in a dataset with estimated values.As the BMI value is not random so we follow mean to imputing value of bmi to get estimated value.

```
[ ]  1 #imputed mean value as bmi is not random and only small percentage of entries were missing
     2 stroke['bmi'].fillna(stroke['bmi'].mean(), inplace=True)
```

After applying mean process we eliminated the bmi null value.

```
]   1 stroke.isnull().sum()

id                   0
gender               0
age                  0
hypertension         0
heart_disease        0
ever_married         0
work_type            0
Residence_type       0
avg_glucose_level    0
bmi                  0
smoking_status       0
stroke               0
dtype: int64
```

Encoding:

It's a method for working with categorical values. A few values containing object data type were discovered. In our code, we found some objects and int values and we eliminated object values from our coding.

Before encoding:

```
[ ]   1 stroke.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5110 entries, 0 to 5109
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   id                 5110 non-null   int64  
 1   gender             5110 non-null   object 
 2   age                5110 non-null   float64
 3   hypertension       5110 non-null   int64  
 4   heart_disease      5110 non-null   int64  
 5   ever_married       5110 non-null   object 
 6   work_type          5110 non-null   object 
 7   Residence_type     5110 non-null   object 
 8   avg_glucose_level  5110 non-null   float64
 9   bmi                4909 non-null   float64
 10  smoking_status     5110 non-null   object 
 11  stroke             5110 non-null   int64  
dtypes: float64(3), int64(4), object(5)
memory usage: 479.2+ KB
```

After encoding:

```
1 from sklearn.preprocessing import LabelEncoder
2
3 # the LabelEncoder object
4 livingstone = LabelEncoder()
5 #
6 #stroke['gender'] = livingstone.fit_transform(stroke['gender'])
7 #stroke['ever_married'] = livingstone.fit_transform(stroke['ever_married'])
8 ##stroke['work_type'] = livingstone.fit_transform(stroke['work_type'])
9 #stroke['Residence_type'] = livingstone.fit_transform(stroke['Residence_type'])
10 stroke['smoking_status'] = livingstone.fit_transform(stroke['smoking_status'])
11 stroke.head()
12
```

	age	hypertension	heart_disease	avg_glucose_level	bmi	smoking_status	stroke
0	67.0	0	1	228.69	36.600000	1	1
1	61.0	0	0	202.21	28.893237	2	1
2	80.0	0	1	105.92	32.500000	2	1
3	49.0	0	0	171.23	34.400000	3	1
4	79.0	1	0	174.12	24.000000	2	1

```
[ ]  1 stroke['gender'].unique()
```

```
array(['Male', 'Female', 'Other'], dtype=object)
```

```
[ ]  1 stroke['Residence_type'].head() # just to check
     2
     3
     4
```

```
0    Urban
1    Rural
2    Rural
3    Urban
4    Rural
Name: Residence_type, dtype: object
```

```
[ ]  1 stroke.head()
```

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

Feature Scaling

A preprocessing method called feature scaling is used in machine learning to standardize or normalize a dataset's range of independent variables or features. By ensuring that all of the characteristics have the same scale, it will be impossible for one feature to dominate over another and the optimization process will be more stable and efficient. Here first we select features.

```
1 #feature selection
2 #feature selection
3
4 #stroke = stroke.drop(['id'], axis = 1)
5 stroke = stroke.drop(['id','gender','ever_married','work_type','Residence_type'], axis = 1)
6 stroke.head()
```

	age	hypertension	heart_disease	avg_glucose_level	bmi	smoking_status	stroke
0	67.0	0	1	228.69	36.6	formerly smoked	1
1	61.0	0	0	202.21	NaN	never smoked	1
2	80.0	0	1	105.92	32.5	never smoked	1
3	49.0	0	0	171.23	34.4	smokes	1
4	79.0	1	0	174.12	24.0	never smoked	1

Scaling :

Here we remove feature from feature selection as we can see we remove ‘stroke’ from the features and then we reduced the values of other features so that others cannot dominate one another.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler

# Scaling and splitting

ba = stroke.drop('stroke', axis=1)
scale = StandardScaler()
scaled_target = scale.fit_transform(ba)
scaled_set = pd.DataFrame(scaled_target, columns=ba.columns)
scaled_set.head()

```

	age	hypertension	heart_disease	avg_glucose_level	bmi	smoking_status
0	1.051434	-0.328602	4.185032	2.706375	1.001234e+00	-0.351781
1	0.786070	-0.328602	-0.238947	2.121559	4.615554e-16	0.581552
2	1.626390	-0.328602	4.185032	-0.005028	4.685773e-01	0.581552
3	0.255342	-0.328602	-0.238947	1.437358	7.154182e-01	1.514885
4	1.582163	3.043196	-0.238947	1.501184	-6.357112e-01	0.581552

Dataset Splitting

In machine learning, "splitting" refers to the process of dividing a dataset into two parts.For model construction and assessment, a 70%–30% train-test split has been used. This section makes sure that the machine learning model is trained using 70% of the dataset, which enables efficient identification of patterns and relationships in the data. The remaining 30 percent is set aside to evaluate the model's generalization ability on hypothetical data, providing a reliable assessment metric.

```
#splitt
from sklearn.model_selection import train_test_split
X = scaled_set
y = stroke['stroke']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.30, random_state=42, stratify=y)
```

Model Training and Testing

We use 3 significant models. KNN, SVM and Logistic Regression models we are using.

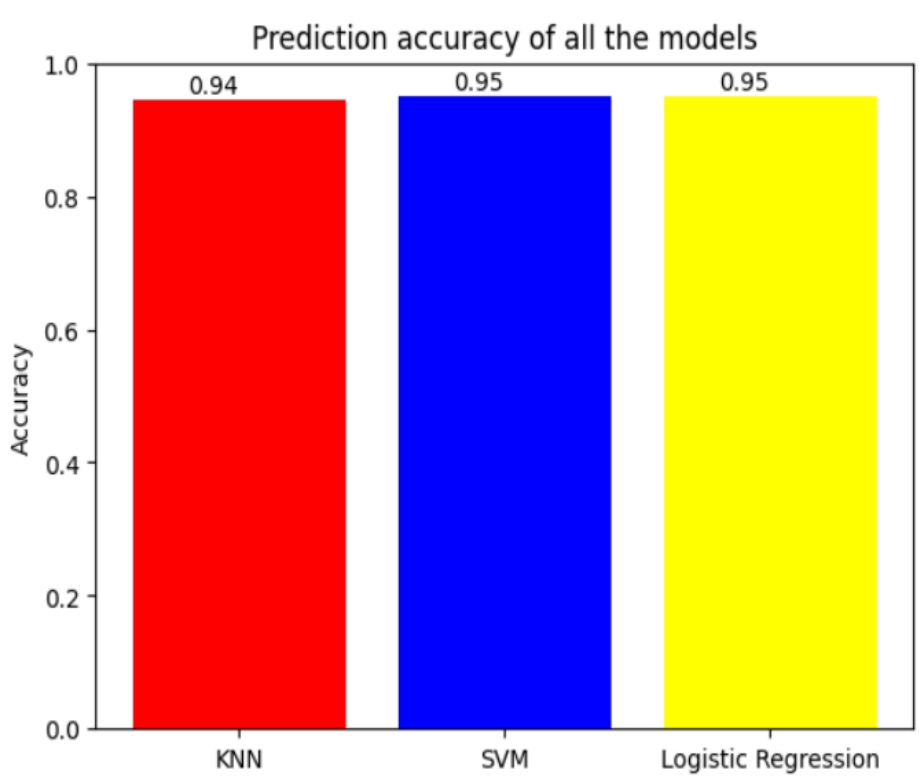
KNN(K-Nearest Neighbors): It's a simple and versatile algorithm that is based on the idea of proximity. KNN is a non-parametric, lazy-learning algorithm, meaning that it doesn't make any assumptions about the underlying data distribution and it doesn't explicitly learn a model during the training phase. Instead, it stores the entire training dataset and makes predictions by finding the "k" training examples that are closest to the input data point in the feature space.

Support Vector Machine (SVM) : Support Vector Machine (SVM) is a supervised machine learning algorithm that can be used for both classification and regression tasks. SVM is particularly effective in high-dimensional spaces and is widely used in various applications, including image classification, text classification, and bioinformatics. The key idea behind SVM is to find a hyperplane that best separates the data into different classes.

Logistic Regression: Logistic Regression is a supervised machine learning algorithm primarily used for binary classification problems, where the target variable has two possible outcomes. Despite its name, logistic regression is a classification algorithm, not a regression algorithm.

Model Selection

Prediction Acccuracy:



Here we can see SVM and Logistic Regression gives the more accurate value than KNN. But the three models are giving almost same value as we can see from the accuracy graph.

Precision and Recall:

Precision is the ratio of correctly predicted positive observations to the total predicted positives. In other words, it measures the accuracy of the positive predictions made by the model.

Whereas, Recall is the ratio of correctly predicted positive observations to the total actual positives. It measures the model's ability to capture all the positive instances.

```python
#precision and recall

from sklearn.metrics import precision_score, recall_score
knn_precision = precision_score(y_test, knn_prede)
knn_recall = recall_score(y_test, knn_prede)

svm_precision = precision_score(y_test, svm_prede)
svm_recall = recall_score(y_test, svm_prede)

lr_precision = precision_score(y_test, lr_prede)
lr_recall = recall_score(y_test, lr_prede)

print(f"KNN Precision: {knn_precision:.2f}, Recall: {knn_recall:.2f}")
print(f"SVM Precision: {svm_precision:.2f}, Recall: {svm_recall:.2f}")
print(f"Logistic Regression Precision: {lr_precision:.2f}, Recall: {lr_recall:.2f}")
```

Confusion Matrix:

In machine learning, a confusion matrix is a table that lists the results of a classification method. There is a breakdown of the amount of false positives (incorrectly anticipated positive instances), false negatives (incorrectly forecasted negative instances), and true positives (correctly predicted positive instances).

By examining the confusion matrix, we can understand where a model excels and where it may falter, helping us fine-tune and optimize the model for better overall performance.

```python
#confusion matrix
from sklearn.metrics import confusion_matrix
mat_knn=confusion_matrix(knn_prede, y_test)
print("confusion matrix of KNN:")
print(mat_knn)
mat_SVM=confusion_matrix(svm_prede, y_test)
print("confusion matrix of SVM:")
print(mat_SVM)
mat_lr=confusion_matrix(lr_prede, y_test)
print("confusion matrix of logistic regression:")
print(mat_lr)

```

Conclusion:

In summary, our mission to develop a machine-learning model for stroke prediction stemmed from a personal tragedy. The dataset, sourced from Kaggle, provided valuable insights into stroke-related factors. Through meticulous preprocessing, including handling null values and encoding categorical data, we prepared the dataset for model training. Three models—K-Nearest Neighbors (KNN), Support Vector Machine (SVM), and Logistic Regression—were employed, with SVM and Logistic Regression exhibiting higher accuracy, precision, and recall than KNN. This collaborative effort represents a journey from personal grief to positive change, aiming to empower individuals and healthcare professionals for proactive stroke prevention through accurate predictions.